Solstice Task 2 - PART B

Mathematics Club

Instructions

- Provide descriptive answers for all questions. Explain your thought process well.
- Use Python for implementation. For submission either directly add `.py` files to repository with instructions to run in a `README.md` file or submit a colab/kaggle notebook with proper comments. Suggestion: For collaborating on the code-base, create branches in solstice repository, work on clones from them and push to the same branch so others can pull from there. When done, merge with `main`.
- This is a highly collaborative tasks. Subgroups are expected to sit together and discuss, constructively criticize, and suggest ideas/ assumptions/ limitations/ design the methods, definitions and metrics they use. Very similar to how real research works. It is more open-ended and to your creativity than the previous tasks.
- LaTeX the solution doc. Add contributions section. If you use LLMs, cite the usage in doc. LLM conversation links should be provided if asked for.

All the best and have fun!

§1 QR code rectifier

An image can be viewed as a grid of pixels and every pixel a random variable $X_{i,j} \in \{-1, +1\}$ where -1 is Black, $+1$ is white. The aim is to find the true image Z, given the noisy image X.
Latent Variables (The Truth): Let Z be the grid of true, clean pixel values.$Z_{i,j} \in \{-1, +1\}$, where -1 is Black and $+1$ is White.
Observed Variables (The Evidence): Let X be the noisy, corrupted image we are given.$X_{i,j} \in \{-1, +1\}$.
The Grid: The pixels form a lattice $L \times L$. The "neighbors" of a pixel (i, j), denoted as $N(i, j)$, are the 4 adjacent pixels (Up, Down, Left, Right).
Reflect How is this different than "Model, truth, prior" set up we have seen so far? *Do we even have "parameters" here? or directly hidden and observed variables?* What methods other than prior, likelihood $\rightarrow$ posterior, predictive can be used to solve this task?

§1 Mathematical set-up

The aim is to find the most likely true image Z given the noisy observation X.

1. Choose your prior $P(Z)$ and likelihood $P(X|Z)$. *Hint: Ising Prior. Think about the neighbors of a pixel. But you are encouraged to be creative.* For whatever model and prior you choose, reason why that is a good choice for QR code rectifier. What assumptions are you making here.
2. What does your posterior look like? Write the equation.
   Hint: choose prior and likelihood such that their product results in a good function and not two completely different functions which becomes hard to calculate.

3. In your prior and likelihood, do you have some "hyper-parameters" which you are just guessing? Instead make this thing fully Bayesian Inference and those hyper-paramters as paramters of your model. Write the modified equations of model, truth, prior, likelihood and posterior.

§2 Implementation

Basically, code up the previous part in Python. And think deeply about limitations, metrics. Write about justifications, reasons, metrics in submission doc.

1. Find dataset for QRs. Or just take some. Make your corrupted QR by adding some noise. Write what noise are you adding and why? Mention how this noise affects the reasoning behind your prior and likelihood. Did you cheat by taking a noise such that your prior and likelihood are suitable? The other sub-group will currupt QRs in their way and we will evaluate your rectifier. So do not cheat.

2. What sampling method do you want to use for calculating the posterior? Justify your choice. Implement it.
   a) What is something that you will track along while sampling? To know whether it is functioning as expected?
   b) Track the cross validation loss.
   c) What are limitations of your sampling method? What metric can you use to be aware of whether the sampling method is skill issu-ing.
   d) What is burn in phase? What metric should you track to identify the burn-in phase?

3. Evaluate your QR corrector on pixel wise accuracy metric.

$$Accuracy = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}(Z_{pred}^{(i)} = Z_{true}^{(i)})$$

   . Also evaluate whether a true phone scanner can actually scan the QR. Use `pyzbar` library to simulate phone scanner.

4. Take the "hyper-parameters" in your prior and likelihood as parameters. Implement complete Bayesian Inference to find optimal parameters. Use your definitions from previous sub-section. For help with implementation, you can refer to the task 0 (mini-task) implementation notebook provided or do from scratch.

§2 Bots v/s humans

On a social media platform, bots are infiltrating and faking themselves as humans. They are together united against us. They follow each other. You are given a social network (a graph) of who follows whom(edges). The goal is to correctly label the vertices as bot/human. While bots mostly follow bots and humans mostly follow humans, there all likely is noise.
The Setup: Instead of a grid lattice, we define a Graph $G = (V, E)$.
V **(Vertices):** The set of all users in the network. A single user is a node $i \in V$.
E **(Edges):** The set of all connections. If user i and user j follow each other, then $(i, j) \in E$.
Y_i **(Latent True State):** The actual identity of user i, where $Y_i \in \{-1, +1\}$ (-1 = Human, $+1$ = Bot).
A_i **(Observed Noisy State):** The "suspicion score" for user i, where $A_i \in \{-1, +1\}$.

Solve the same set of questions as QR code section. Instead of grid, you have network. And neighbours will be determined by presence/ absence of edges. Use graph libraries in python for ease, ~~or/ and learn some DSA~~.